Student Representatives

- Sneha Sunil
  - sneha.sunil@student.unimelb.edu.au
- Shannen Latisha
  - shannen.latisha@student.unimelb.edu.au
- Haoze (Alex) Li
  - haoze.li9@student.unimelb.edu.au

More information about our student representatives can be found in:

Canvas → Modules → Welcome and Subject Overview → Meet the Staff.

COMP90038 Algorithms and Complexity

Brute Force Methods

Junhao Gan

Week 3 Lecture 5

Semester 1, 2026

Brute Force Algorithms

Straightforward problem solving approach, usually based directly on the problem definitions.

Exhaustive search for solutions is a prime example.

- Selection sort
- String matching
- 2D Closest pair
- Exhaustive search for combinatorial solutions
- Graph traversal

The Sorting Problem

The Sorting Problem: Given a list of n *orderable* items, sort these items in a *non-descending* order.

For simplicity, you may imagine that these items are *integers*. But keep in mind that this is *not* a requirement.

Question: What kind of brute-force algorithms can you think of?

- By definition, a sorted order of the items is essentially a *permutation* of them.
- Thus, we may try *every possible permutation* of the items and then check whether it is sorted.
- What would be the time complexity of this algorithm? $O(n \cdot n!)$

As we mentioned in the previous lectures, better brute-force approaches exist.

Example: Selection Sort

We already saw this algorithm:

```
function SelSort(A[·], n)
    for i ← 0 to n − 2 do
        min ← i
        for j ← i + 1 to n − 1 do
            if A[j] < A[min] then
                min ← j
        t ← A[i]                          ▷ Swap A[i] and A[min]
        A[i] ← A[min]
        A[min] ← t
```

The complexity is $O(n^2)$.

We shall soon meet better sorting algorithms.

Some Properties of Sorting Algorithms

A sorting algorithm is

- in-place if it does not require additional memory except, perhaps, for a few units of memory.
- stable if it preserves the relative order of elements that have identical keys.
- input-insensitive if its running time is fairly independent of input properties other than size.

Some Properties of Selection Sort

In-place?

Stable?

Input-insensitive?

While the running time bound is quadratic, selection sort makes only about n exchanges, i.e., item swaps.

So: It is a good algorithm for sorting small collections of *large* items.

The String Matching Problem

Given:

Pattern p: a string of m characters to search for, and

Text t: a long string of n characters to search in,

the goal of the **String Matching Problem** is to find the *first occurrence* (if exists) of the pattern p in the text t.

For example: $t =$ "Hey man, how are you doing?", and $p =$ "war"

How about for $p =$ "w ar"?

The basic idea is to check *every possible m-length sub-string* in the text t *starting with each character in t* to see whether it matches the pattern p or not.

Brute Force String Matching

We use i to run through the text, and j to run through the pattern.

for $i \leftarrow 0$ to $n - m$ **do**
    $j \leftarrow 0$
    while $j < m$ and $p[j] = t[i + j]$ **do**
        $j \leftarrow j + 1$
    if $j = m$ **then**
        return i
return -1

Analysing Brute Force String Matching

For each of $n - m + 1$ positions in t, we make up to m comparisons.

Assuming n is much larger than m, this means $O(mn)$ comparisons.

However, in general, this algorithm performs better than what the theoretical bound suggests, because for a *dis-match*, we can stop the rest comparisons right away.

Therefore, for many purposes, this brute-force algorithm is acceptable.

There are better algorithms, in particular for smaller *alphabets* such as **binary strings** or strings of **DNA nucleobases**.

Later we shall see more sophisticated string matching algorithms.

The Closest Pair Problem

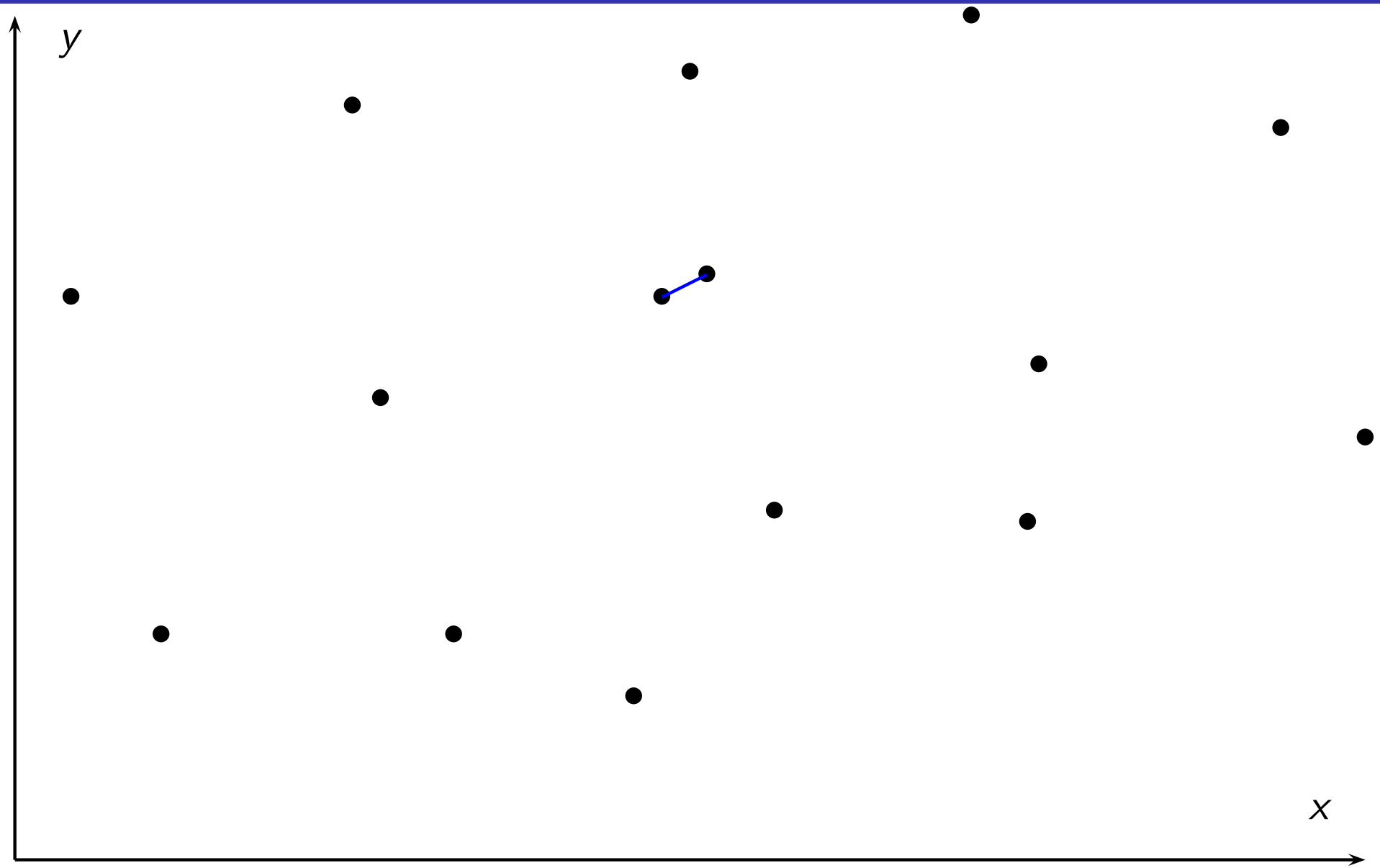
Consider two points $p_1 = (a_1, a_2, \ldots, a_d)$ and $p_2 = (b_1, b_2, \ldots, b_d)$ in d-dimension space; the *Euclidean Distance* between p_1 and p_2 is computed as:

$$dist(p_1, p_2) = \sqrt{\sum_{i=1}^{d} (a_i - b_i)^2}\,.$$

The 2-Dimensional Closest Pair Problem:

Given a set S of n points in 2-dimensional space, the goal is to find a pair of points p_1^* and p_2^* in S such that $dist(p_1^*, p_2^*)$ is *no more than* the distance between any other pair of points in S.

The 2D Closest Pair Problem



Brute Force Geometric Algorithms: Closest Pair

Try out all combinations $p_i = (x_i, y_i)$ and $p_j = (x_j, y_j)$ with $i < j$:

$min \leftarrow \infty$
for $i \leftarrow 0$ to $n - 2$ **do**
    for $j \leftarrow i + 1$ to $n - 1$ **do**
        $dist \leftarrow sqrt((x_i - x_j)^2 + (y_i - y_j)^2)$ ▷ Distance for this pair
        if $dist < min$ **then**
            $min \leftarrow dist$ ▷ Smallest distance so far
            $p_1^* \leftarrow i$ ▷ Remember this (i,j) combination
            $p_2^* \leftarrow j$
return p_1^*, p_2^*

Analysing the Closest Pair Algorithm

It is not hard to see that the algorithm is $O(n^2)$.

Later we shall see how a clever divide-and-conquer approach leads to a $O(n \log n)$ algorithm for this problem.

Brute Force Summary

Simple, easy to program, widely applicable.

Standard approach for small tasks.

Reasonable algorithms for some problems.

But: Generally inefficient—does not scale well.

Use brute force for prototyping, or when it is known that input remains small.

Exhaustive Search

Exhaustive Search

Problem type:

- Combinatorial decision or optimization problems
- Search for an element with a particular property
- Domain grows exponentially, for example all permutations

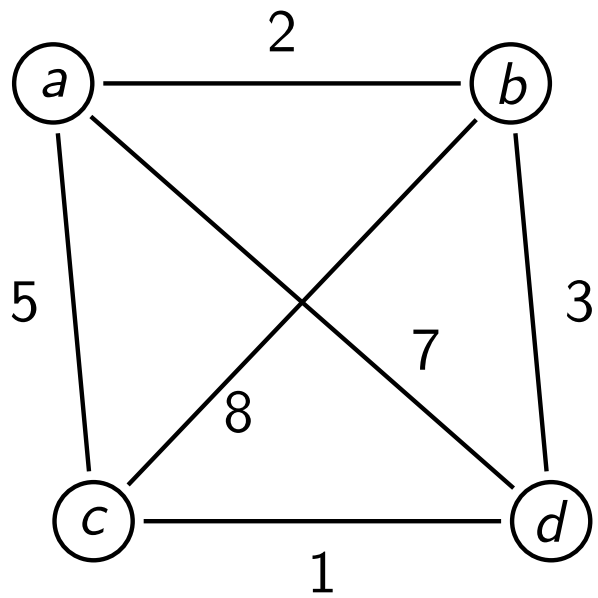
The brute-force approach—generate and test:

- Systematically construct all possible solutions
- Evaluate each, keeping track of the best so far
- When all potential solutions have been examined, return the best found

Example 1: Travelling Salesman Problem (TSP)

Given a *weighted undirected* graph $G = (V, E)$, find the *shortest tour* (in terms of the edge weight sum) which

- starts and ends at a same node, and
- visits every other node in G *once and exactly once*.



$a - b - c - d - a$	:	18
$a - b - d - c - a$	:	11
$a - c - b - d - a$	:	23
$a - c - d - b - a$	:	11
$a - d - b - c - a$	:	23
$a - d - c - b - a$	:	18

Example 2: Knapsack

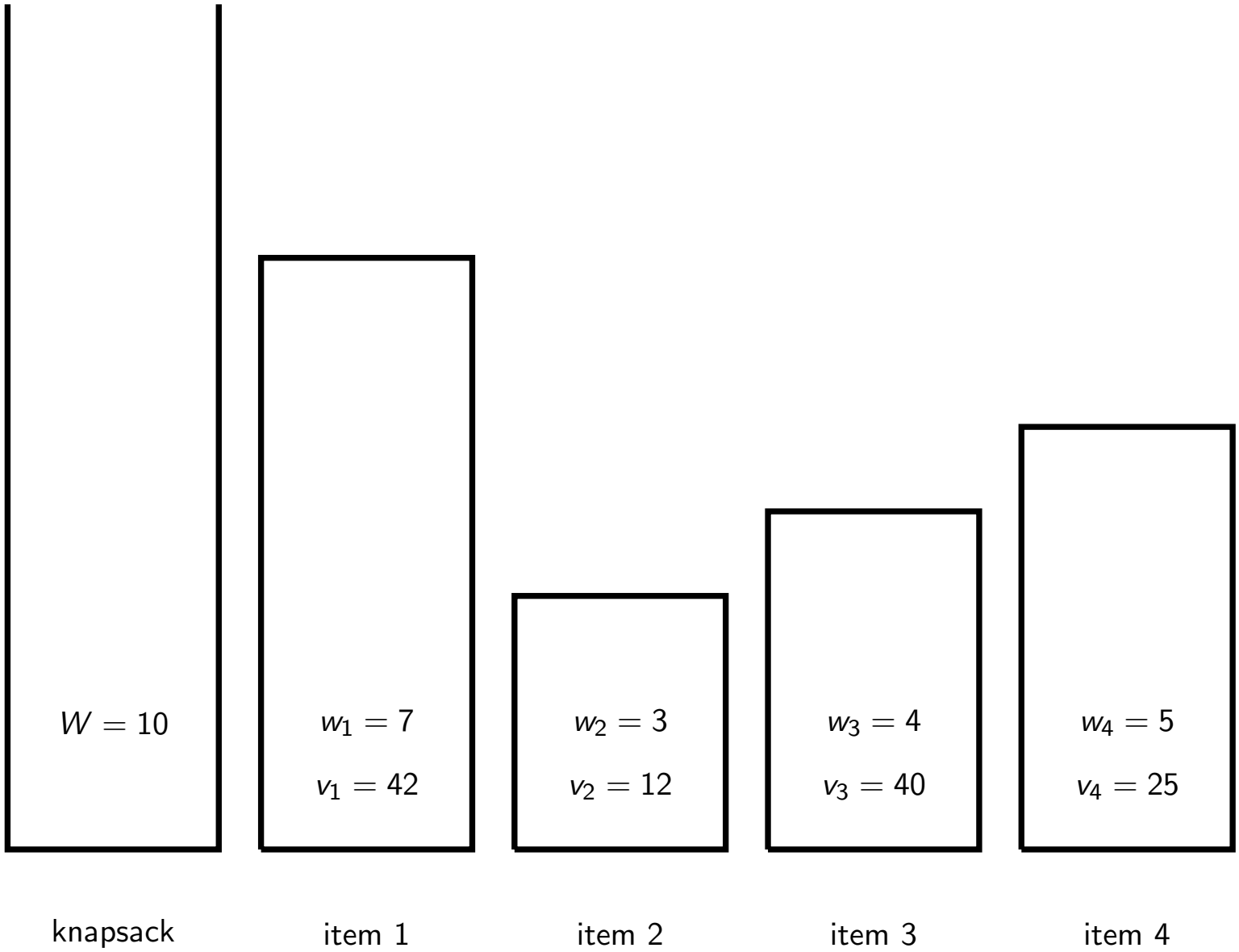
Given n items with

- weights: $w_1, w_2, \ldots, w_n$
- values: $v_1, v_2, \ldots, v_n$
- knapsack of capacity W

the goal of the **Knapsak Problem** is to find a set S^* of items such that:

- the *total weight sum* of the items S^* is at most W (that means, they fit in the knapsack), and
- the *total value sum* of the items S^* is *maximized*.

Example 2: Knapsack



Example 2: Knapsack

Set	Weight	Value
$\emptyset$	0	0
$\{1\}$	7	42
$\{2\}$	3	12
$\{3\}$	4	40
$\{4\}$	5	25
$\{1,2\}$	10	54
$\{1,3\}$	11	NF
$\{1,4\}$	12	NF

Set	Weight	Value
$\{2,3\}$	7	52
$\{2,4\}$	8	37
$\{3,4\}$	9	65
$\{1,2,3\}$	14	NF
$\{1,2,4\}$	15	NF
$\{1,3,4\}$	16	NF
$\{2,3,4\}$	12	NF
$\{1,2,3,4\}$	19	NF

NF means "not feasible": exhausts the capacity of the knapsack.

Later we'll find a better algorithm based on dynamic programming.

Comments on Exhaustive Search

Exhaustive search algorithms have acceptable running times only for very small instances.

But for some problems, it is known that there is essentially no better alternative.

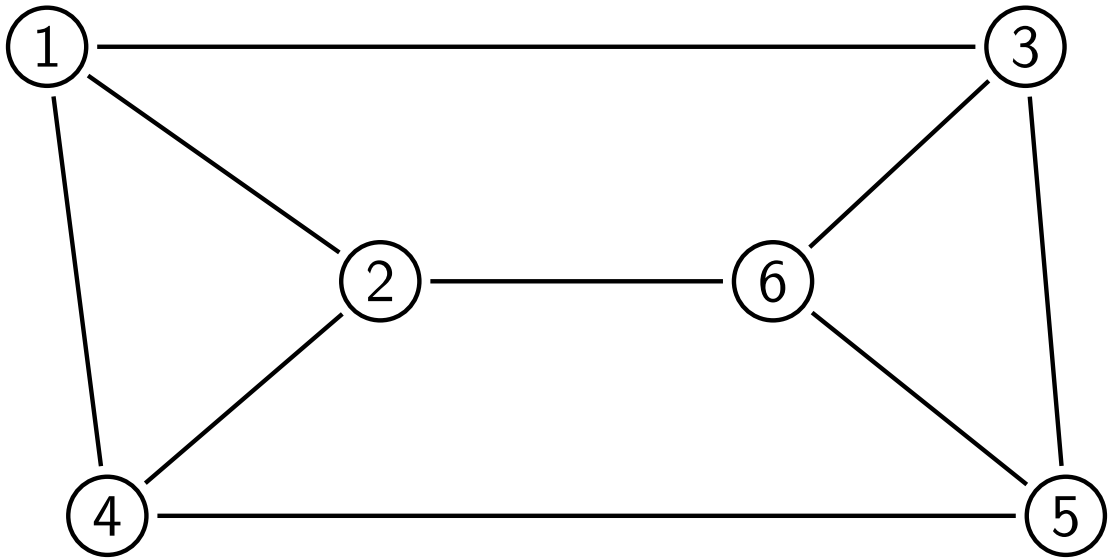
For a large class of important problems, it appears that there is no better alternative, but we have no proof either way.

Hamiltonian Tours

The **Hamiltonian Tour Problem** is this:

Given an *undirected* graph, decide if there is a *simple tour* (a path that visits each node once and exactly once, except it returns to the starting node)?

Is there a Hamiltonian tour of this graph?

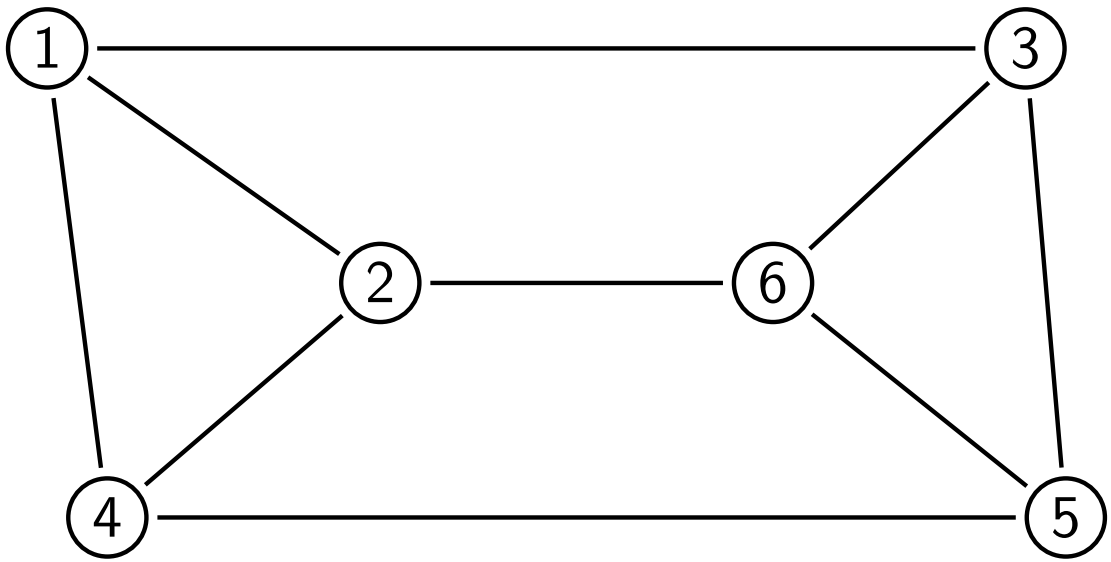


Eulerian Tours

The **Eulerian Tour Problem** is this:

Given an *undirected* graph, decide if there is a tour which visits each edge once and exactly once and returns back to the starting node?

Is there a Eulerian tour of this graph?



Hard and Easy Problems

Recall that by a problem we usually mean a parametric problem: an infinite family of problem "instances".

Sometimes our intuition about the difficulty of problems is **not** very reliable. The Hamiltonian Tour problem and the Eulerian Tour problem look very similar, but one is hard and the other is easy.

We shall see more examples of this phenomenon later.

Hard and Easy Problems

For many important optimization problems, we do not know of solutions that are essentially better than exhaustive search (a whole raft of NP-complete problems, including TSP, knapsack).

In those cases we may look for approximation algorithms that are fast and still find solutions that are reasonably close to the optimal.

We will return to this idea in Week 12 if time permits.